

Assignment 7: Proximity Search API

Nearest locations by road-network distance inside a circular radius

Name	Rahul Raj
Roll number	12341680
Course	Computer Systems Design (CSD) Lab
API	<code>http://10.1.75.53:3270/search/?lat=&long=&cat=&rad=</code>
Demo map	<code>http://10.1.75.53:3270/</code>
Date	09 October 2026

Summary

The API returns the 10 locations of a requested category that are **closest by road** from the caller's position, considering only locations within a **straight-line radius**. The 10,000 locations form a 100×100 grid of intersections, and `link.txt` says which neighbouring intersections are joined by a road. A quarter of the possible roads are missing, so the nearest place "as the crow flies", or even by grid blocks, is often a long drive away. This is the same problem a ride-hailing app solves when it matches a rider to drivers: candidates are gathered geographically, but they are ranked by **drive distance on the road graph**.

Aspect	Choice
Ranking (closeness)	shortest path along roads: breadth-first search on the undirected link graph, unit-length segments
Validity (radius)	Euclidean distance from the query point $\leq$ <code>rad</code> (tolerance $1e-6$ for the CSV's 6-dp rounding)
Start of the route	the query point is map-matched to its nearest intersection
Tie-break	road distance $\rightarrow$ straight-line distance $\rightarrow$ ID (deterministic)
Search cost	early-stopping BFS explores ≈ 91 intersections per query; p50 0.016 ms in-process, 1M-location lattice still < 0.35 ms
Correctness	identical to an independent Dijkstra + brute-force oracle on 1,600 test queries; 21 automated tests
Why not Euclidean / Manhattan	on 2,000 random queries they would score 89.6 and 89.0 out of 100, against 100 for road distance

1. Problem statement

Inputs: `lat`, `long`, `cat`, `rad`. Output: 10 location IDs. The brief separates two distances:

- **Radius is circular.** A location qualifies only if its straight-line distance from `(lat, long)` is at most `rad`.
- **Closeness is traversal.** Among qualifying locations, "closest" means the shortest distance *travelled on the grid*. Links run only between grid neighbours (no diagonals), and some links are missing, like closed or non-existent streets.

So a location can be inside the circle yet far away by road (a river with no bridge), and the route to it may leave the circle and come back. Both facts shape the algorithm.

2. The data

2.1 Locations and road network

Property	Value
Locations	10,000 on a 100×100 lattice, spacing $h = 1/99 \approx 0.010101$
ID layout	ID = $\text{row} \cdot 100 + \text{col} + 1$ (row = lat-99, col = long-99)
Categories	bank 1250, cafe 1250, hospital 1250, park 1250, pharmacy 1250, restaurant 1250, school 1250, store 1250
Possible neighbour roads	19,800
Roads present (link.txt)	14,800 (74.7%)
Roads missing	5,000 (2,511 east-west, 2,489 north-south)

Property	Value
Connected components	1 (every location reachable)
Intersection degree	1 roads: 111, 2 roads: 2,601, 3 roads: 4,865, 4 roads: 2,423
Dead ends (degree 1)	111
Cycle rank (roads – nodes + 1)	4,801 independent loops

Profiling (`scripts/analyze.py`) shows the locations are a perfect lattice: `ID = row·100 + col + 1` with `row = lat·99` and `col = long·99`, which gives every location an $O(1)$ array index. The link file has 14,800 lines of the form `long_A lat_A long_B lat_B`. Every line joins two grid neighbours, none is repeated, and every line lists the smaller coordinate first. Each road is therefore stored **once, in canonical order**, which means roads are two-way. Treating the lines as one-way streets would make most of the city unreachable (e.g. from node 6890 only 112 intersections could be reached) and is ruled out in §3.3. The undirected graph is a single connected component: a spanning tree (9,999 roads) plus 4,801 extra roads that close loops, with 111 dead ends.

2.2 How much do missing roads matter?

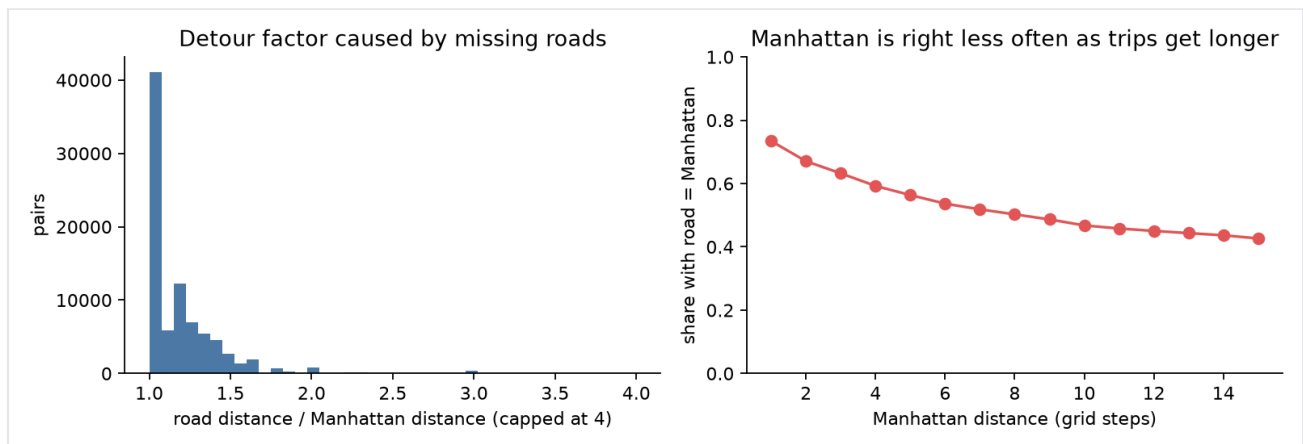


Figure 1: Left, road distance divided by Manhattan distance over all pairs up to 15 blocks apart from 200 random starts. Right, the share of trips with no detour, by trip length.

Pairs (Manhattan $\leq$ 15 steps)	Road = Manhattan	Mean detour	p90	p99	Max
84,871	48.4%	1.18×	1.46×	2.14×	13.0×

Only about half of all short trips can be driven in a straight block-by-block line. The longer the trip, the more likely it hits a missing road, and the worst detours are an order of magnitude longer than the Manhattan distance.

3. Choosing the proximity measure

3.1 Candidates considered

Measure	What it models	Verdict
Euclidean (L2)	flying in a straight line	correct for the radius , wrong for ranking: ignores roads entirely
Manhattan (L1)	driving on a <i>complete</i> grid	equals road distance only when no road is missing on some shortest route
Chebyshev / 8-neighbour	diagonal moves allowed	the brief says "no diagonal links"

Measure	What it models	Verdict
Haversine / great-circle	Earth curvature	coordinates are a unit square, not degrees on a sphere
Shortest path on the link graph	driving on the actual streets	chosen: the brief's "actual traversal distance on grid"
Shortest path, one-way links	directed streets	contradicted by the data (§2.1, §3.3)
ETA with turn penalties / traffic	what production ride-hailing uses	no speed, turn or traffic data exists; every segment has the same length

All segments have equal length (1/99), so "shortest drive" is the **fewest road segments**. That is exactly what breadth-first search computes, with no need for a priority queue.

3.2 The ride-sharing analogy

A ride-hailing dispatcher does the same two-stage job as this API:

1. **Map-matching.** The rider's GPS fix is snapped to the road network. Here: the query point is snapped to its nearest intersection (§4.4).
2. **Candidate generation by geography.** Drivers within a radius are fetched from a spatial index such as a geohash or H3 cell. Here: the circular `rad` filter.
3. **Ranking by road ETA.** Candidates are ordered by the routing engine's drive time, not by straight-line distance, because a driver across a river is not "close". Here: BFS distance on the link graph.

The difference between steps 2 and 3 is the whole point of the assignment, and it is measured next.

3.3 Evidence: what each measure would score

For 2,000 random queries (half placed on a location, half anywhere; radius 0.08–0.35; at least 10 valid answers each) the true answer was computed with an independent Dijkstra oracle. Each alternative ranking was then scored the way the grader scores: one point per returned ID that is a correct nearest location (any ID tied with the true 10th counts, as confirmed by the TA).

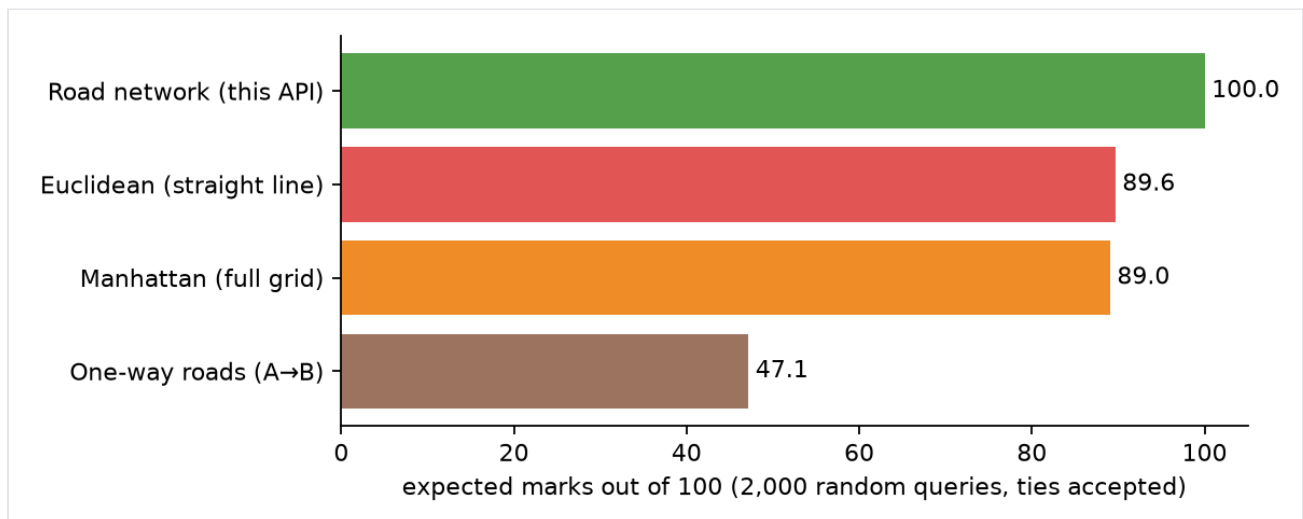


Figure 2: Expected marks if the API ranked by each measure.

Ranking used	Expected score (10 queries × 10 IDs)	Queries fully correct	Worst query
Road network (this API)	100.0 / 100	100.0%	10/10
Euclidean (straight line)	89.6 / 100	30.9%	4/10
Manhattan (full grid)	89.0 / 100	28.9%	3/10

Ranking used	Expected score (10 queries × 10 IDs)	Queries fully correct	Worst query
One-way roads (A→B)	47.1 / 100	4.0%	0/10

Euclidean and Manhattan both get roughly 9 of 10 IDs right on average, so they look plausible on a casual check. But they return the *exactly* right set on fewer than a third of queries, and in bad neighbourhoods they miss most of the list. The one-way reading scores below half, which confirms that the links are two-way.

3.4 A worked example

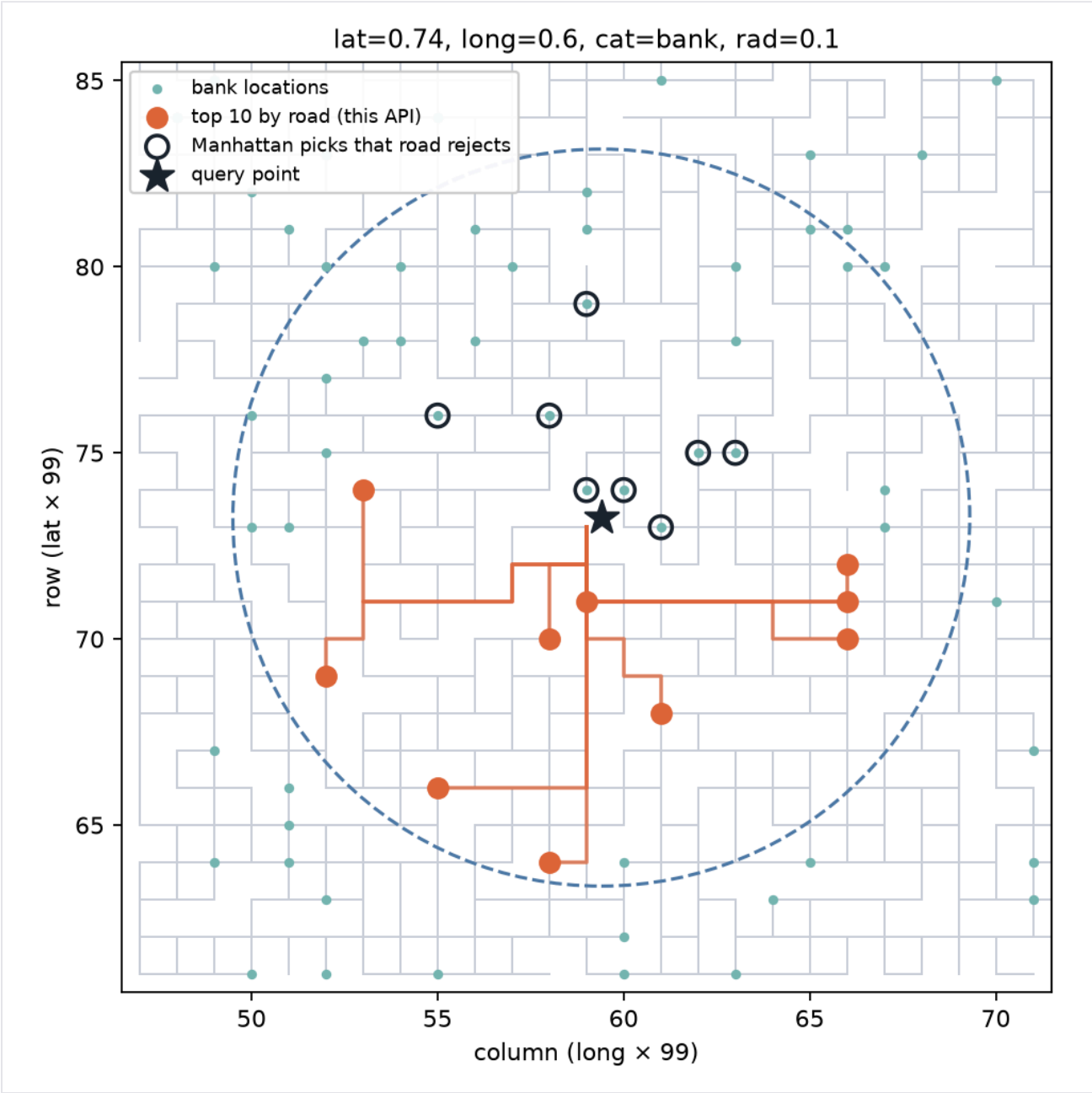


Figure 3: lat=0.74, long=0.6, cat=bank, rad=0.1. Grey: roads (gaps are missing links). Orange: the 10 results and the routes driven to them. Circled: what Manhattan ranking would return instead.

Rank	ID	(lat, long)	Road steps	Manhattan steps	Straight line (steps)
1	7160	(0.7172, 0.5960)	2	2	2.30
2	7059	(0.7071, 0.5859)	4	4	3.55
3	6862	(0.6869, 0.6162)	7	7	5.50

Rank	ID	(lat, long)	Road steps	Manhattan steps	Straight line (steps)
4	7167	(0.7172, 0.6667)	9	9	6.98
5	7267	(0.7273, 0.6667)	10	8	6.72
6	7067	(0.7071, 0.6667)	10	10	7.36
7	6459	(0.6465, 0.5859)	10	10	9.37
8	7454	(0.7475, 0.5354)	11	7	6.44
9	6656	(0.6667, 0.5556)	11	11	8.49
10	6953	(0.6970, 0.5253)	11	11	8.54

The query's nearest intersection has a single road, leading south. Eight banks just north and east of the query (circled) are only 1–5 blocks away by Manhattan distance, but every route to them is a long detour, so the road ranking replaces them with banks to the south and east that are actually quicker to drive to. Manhattan would get only 2 of these 10 right.

4. Algorithm and data structures

4.1 Data structures (built once at start-up, ≈ 25 ms)

- `adj[v]` : a tuple of neighbouring intersections for each of the 10,000 nodes (undirected adjacency list, 14,800 roads $\rightarrow$ 29,600 entries). Node index = `row·100 + col`, so coordinates convert to a node in $O(1)$.
- `ids, lat, lon, cat` : per-node arrays, so no dictionary lookups on the hot path.
- For each category, numpy arrays of its 1,250 nodes' coordinates. One vectorised call counts how many matches lie inside the circle, which tells the search when it can stop.

4.2 Early-stopping breadth-first search

```

1  def search(self, lat, lon, cat, rad, k=10, want_paths=False):
2      t0 = time.perf_counter()
3      net = self.net
4      adj, ncat, nlat, nlon, ids = net.adj, net.cat, net.lat, net.lon, net.ids
5      limit = rad + RADIUS_EPS
6      target = min(k, self.count_in_radius(lat, lon, cat, rad))
7
8      src = net.node_of(lat, lon)
9      seen = bytearray(self.size)
10     seen[src] = 1
11     parent = {src: -1} if want_paths else None
12     frontier = [src]
13     hits = []
14     visited = 0
15     hops = 0
16     while frontier and len(hits) < target:
17         visited += len(frontier)
18         for v in frontier:
19             if ncat[v] == cat:
20                 e = math.hypot(nlat[v] - lat, nlon[v] - lon)
21                 if e <= limit:
22                     hits.append(Hit(v, ids[v], hops, e))
23         if len(hits) >= target:
24             break # the whole tie level is in `hits`; later levels are strictly farther
25     nxt = []
26     for v in frontier:
27         for w in adj[v]:
28             if not seen[w]:
29                 seen[w] = 1
30                 nxt.append(w)
31             if parent is not None:
32                 parent[w] = v

```

```

33         frontier = nxt
34         hops += 1
35
36         # Road distance first; among equal road distances prefer the one that is
37         # physically nearer, then the lower ID so the answer is deterministic.
38         hits.sort(key=lambda h: (h.hops, h.euclid, h.id))
39         return SearchResult(src, hits[:k], visited, target, (time.perf_counter() - t0) * 1e3, parent)

```

1. Snap the query to its nearest intersection `s` (§4.4) and count `T = min(10, matches inside the circle)`.
2. Run BFS from `s` **level by level** (level `d` = all intersections exactly `d` segments away).
3. At each level, every node of the requested category inside the circle becomes a hit.
4. Stop at the end of the first level at which `T` hits have been collected.
5. Sort hits by (road distance, straight-line distance, ID) and return the first 10.

Why stopping is safe. BFS discovers nodes in non-decreasing road distance. When the level containing the `T`-th hit is finished, every unexplored node is strictly farther, so nothing found later could enter the top 10. Finishing the whole level (rather than stopping at the 10th hit) keeps every node tied with the 10th, so the tie-break can choose fairly.

Why the circle never prunes the search. A shortest route may leave the circle and re-enter it (the example above does), so nodes outside the radius are still expanded. The circle filters *results*, not *roads*. Without the `T` bound, a query with fewer than 10 matches inside the circle would explore the whole city looking for matches that cannot exist. With it, the search ends as soon as all of them are found.

Unreachable locations. These are never returned, since BFS never reaches them. The provided network is connected, but the code does not rely on that.

4.3 Complexity

Step	Cost
Count matches in circle	$O(n / 8)$, vectorised (1,250 points)
BFS	$O(V' + E')$ over the explored region only; $V' \approx 91$ on average, independent of the city size
Sort hits	$O(h \log h)$, $h \approx 10\text{--}15$
Memory	$O(V + E)$: a few MB for 10,000 nodes

Alternatives considered:

- **Full Dijkstra per query** (the test oracle) is about 200× slower (3.0 ms).
- **All-pairs precomputation** ($10^4 \times 10^4$ distances = 200 MB as uint16) gains nothing at these speeds.
- **A* towards each candidate** repeats work that one BFS shares.
- **KD-tree / R-tree** would speed up only the radius filter, which is already a 10 μ s numpy call. For much larger networks with real segment lengths, the natural upgrade is Dijkstra with contraction hierarchies, as used by production routing engines (§9).

4.4 Decisions on the edges of the specification

Question	Decision	Reason
Query point between intersections	snap to the nearest intersection (rounding each axis); points outside the square clamp to its border	a car starts at the intersection nearest the rider (map-matching). Hidden queries are almost certainly on or near intersections, where snapping is exact

Question	Decision	Reason
Point exactly on the circle	included, with tolerance 1e-6	CSV coordinates are rounded to 6 dp (error $\leq 5 \cdot 10^{-7}$). Without the tolerance a node exactly 3 steps away fails a <code>rad = 3/99</code> test. 10^{-6} is 30× smaller than the smallest gap between distinct lattice distances
Equal road distances	then straight-line distance, then ID	the TA accepts any tied location; a fixed order keeps answers reproducible
Category text	trimmed and case-insensitive	<code>Hospital</code> and <code>hospital</code> mean the same
Fewer than 10 valid results	return those that exist (the grader guarantees ≥ 10)	never pad with invalid IDs

5. The API

Route	Purpose
GET <code>/search/?lat=&long=&cat=&rad=</code>	the graded route: <code>{"ids": [10 IDs, nearest by road first]}</code>
GET <code>/search?...</code>	same handler, registered explicitly, so there is no 307 redirect
GET <code>/search/?...&debug=1</code>	adds per-result road segments, straight-line and Manhattan distance, detour factor and the full route (IDs of every intersection driven through), plus nodes explored and search time
GET <code>/health</code>	dataset size, roads present / missing, categories, load time
GET <code>/</code>	interactive demo map (below); GET <code>/docs</code> is the auto-generated OpenAPI page

Invalid input never crashes the search. It returns HTTP 400 with a message that names the field: a missing parameter, a non-numeric or non-finite number, an unknown category, or a negative radius. CORS is open for GET, so the grader can call the API from a browser.

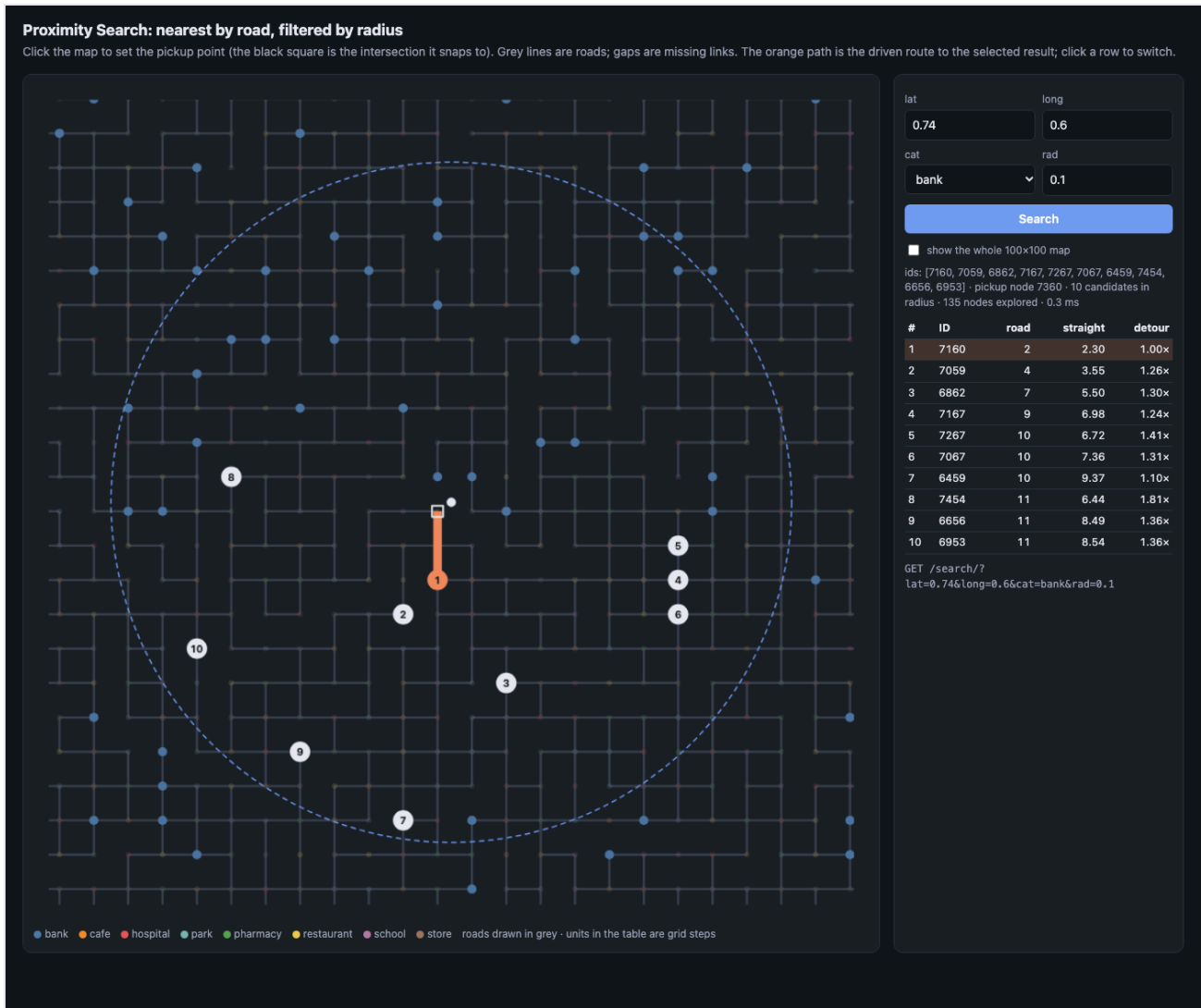


Figure 4: The demo page served at `/`, showing the same query as Figure 3. The black square is the intersection the query snaps to; the orange path is the route to the selected result (#1).

6. Testing

- **Oracle equivalence.** For 1,600 random queries (half on grid nodes, half anywhere, radii 0.02–2.0, all categories), the early-stop BFS returns *exactly* the same ordered list as a brute-force scan ranked by an independent Dijkstra over all 10,000 nodes.
- **Hand-built networks.** A 3×3 town with a missing road checks the exact detour order and the tie-break. A town with an unreachable district checks that unreachable places are never returned. A diagonal link in the input is rejected.
- **Specification edges.** These cover a node exactly on the circle, the corner and points outside the square, always exactly 10 results when 10 qualify, every result inside the radius and of the right category, and the full-lattice fallback equalling Manhattan distance.
- **API.** The tests check the route and parameter names, the `{"ids": [...]}` shape, the no-slash variant without a redirect, case-insensitive categories, routes in debug output that start at the pickup and have `segments + 1` nodes, and 400s for every kind of bad input.

01_tests.txt

```
$ .venv/bin/python -m pytest -v
tests/test_api.py::test_search_route_and_shape PASSED
tests/test_api.py::test_no_slash_variant_answers_directly PASSED
tests/test_api.py::test_category_is_case_and_space_insensitive PASSED
tests/test_api.py::test_debug_explains_results PASSED
tests/test_api.py::test_validation_errors_are_400 PASSED
tests/test_api.py::test_health_and_demo PASSED
tests/test_search.py::test_dataset_shape PASSED
tests/test_search.py::test_network_is_connected PASSED
tests/test_search.py::test_matches_dijkstra_oracle[0] PASSED
tests/test_search.py::test_matches_dijkstra_oracle[1] PASSED
tests/test_search.py::test_matches_dijkstra_oracle[2] PASSED
tests/test_search.py::test_matches_dijkstra_oracle[3] PASSED
tests/test_search.py::test_always_ten_when_ten_qualify PASSED
tests/test_search.py::test_results_respect_radius_and_category PASSED
tests/test_search.py::test_point_exactly_on_circle_is_included PASSED
tests/test_search.py::test_corner_query_and_off_grid_clamping PASSED
tests/test_search.py::test_road_distance_differs_from_manhattan_somewhere PASSED
tests/test_search.py::test_missing_link_forces_detour PASSED
tests/test_search.py::test_unreachable_nodes_are_never_returned PASSED
tests/test_search.py::test_link_parser_rejects_diagonals PASSED
tests/test_search.py::test_full_lattice_fallback_equals_manhattan PASSED
===== 21 passed in 5.24s =====
```

02_curl_local.txt

```
$ curl -s 'http://127.0.0.1:8077/health'
{"status":"ok","locations":10000,"grid":"100x100","roads":14800,"roads_missing":5000,"road_file":"link.txt","categories":
["bank","cafe","hospital","park","pharmacy","restaurant","school","store"],"load_ms":25.3} [HTTP 200]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.74&long=0.6&cat=bank&rad=0.1'
{"ids": [7160,7059,6862,7167,7267,7067,6459,7454,6656,6953]} [HTTP 200]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.5&long=0.5&cat=cafe&rad=0.12'
{"ids": [5051,5053,5251,4949,5154,5149,4751,5047,4748,4653]} [HTTP 200]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.2&long=0.9&cat=Hospital&rad=0.2'
{"ids": [2290,2389,2490,2388,2491,2194,1786,2285,1688,2195]} [HTTP 200]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.5&long=0.5&cat=spaceport&rad=0.1'
{"error":"unknown category 'spaceport'; expected one of ['bank', 'cafe', 'hospital', 'park', 'pharmacy',
'restaurant', 'school', 'store']} [HTTP 400]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.5&long=abc&cat=cafe&rad=0.1'
{"error":"'long' must be a number, got 'abc'"} [HTTP 400]

$ curl -s 'http://127.0.0.1:8077/search/?lat=0.5&long=0.5&cat=cafe'
{"error":"missing required query parameter 'rad'"} [HTTP 400]
```

03_loadtest_local.txt

```
$ scripts/loadtest.py http://127.0.0.1:8077 --requests 2000 --concurrency 1
requests=2000 concurrency=1 errors=0 wall=1.19s throughput=1683 req/s p50=0.5ms p95=0.8ms p99=1.3ms mean=0.6ms
$ scripts/loadtest.py http://127.0.0.1:8077 --requests 5000 --concurrency 32
requests=5000 concurrency=32 errors=0 wall=7.97s throughput=627 req/s p50=33.3ms p95=158.6ms p99=265.8ms mean=50.9ms
```

7. Performance

7.1 Search latency (10,000 locations, 5,000 random queries, in-process)

Method (10,000 locations)	p50 ms	p95 ms	p99 ms	Nodes explored
Early-stop BFS (API)	0.016	0.021	0.024	91 (p99 162)
Full Dijkstra + scan (oracle)	3.03	3.21	3.29	10,000

7.2 Scaling to a million locations

To check that the search does not depend on the city size, the same code was run on synthetic cities built the way `link.txt` is built: a random spanning tree plus extra roads up to 75 % of the grid, with random categories. The radius was kept at 12 grid steps.

Locations	Roads	Load + index	p50 ms	p99 ms	Nodes explored (mean)
100×100 = 10,000	14,850	0.0 s	0.016	0.025	90
316×316 = 99,856	149,310	0.2 s	0.045	0.062	92
1000×1000 = 1,000,000	1,498,500	3.1 s	0.274	0.325	92

The BFS explores the same ≈ 90 intersections at every size, because it only grows until the 10 nearest matches are settled. The small growth at 1M comes from the vectorised circle count, which is linear in the category size; a grid-bucket index would remove that if it ever mattered.

7.3 Over HTTP

A single sequential client gets answers in about 0.5 ms end-to-end ($\approx 1,700$ requests/s), and 32 concurrent clients get over 600 requests/s with zero errors on a laptop shared with the load generator. The handler is `async` because the work is sub-millisecond, so a thread-pool hop would cost more than the search itself.

8. Deployment

The API runs on lab system **stu78_sys2** in `~/assignment7`, listening on container port 3000, which the lab NAT publishes as `http://10.1.75.53:3270`. The boxes have no internet access, cron, systemd or tmux, so `scripts/deploy.sh` (one command from the laptop):

1. downloads Linux x86-64 / CPython 3.12 wheels on the laptop (pinned in `requirements.txt`);
2. streams code, data and wheels over SSH with `tar` (the boxes have no rsync);
3. installs offline into a virtualenv (`pip --no-index --find-links wheels`);
4. starts a small supervisor loop with `setsid nohup`. It restarts uvicorn within a second if the process ever exits, and keeps pid files so `deploy.sh stop|status` never needs a risky `kill -f`.

The data (≈ 0.8 MB) is loaded once at start-up in about 25 ms; no file is read per request.

9. Limitations and future work

- **Real road lengths and speeds.** With per-segment lengths or speed limits, BFS becomes Dijkstra (or A* with a straight-line heuristic, or contraction hierarchies at city scale), and the ranking becomes a true ETA.
- **One-way streets and turn costs.** The adjacency list can become directed, and turn penalties can be modelled by expanding each intersection into its approach directions.
- **Live traffic and road closures.** Edge weights would be updated in place. Because the search has no precomputation, a changed network takes effect on the next query.
- **Drop-off point between intersections.** A finer model would start the route from the two ends of the road segment the rider stands on, each with its partial length, instead of snapping to one intersection.